

Ball.h

```
#pragma once

#include<SFML/Graphics.hpp>

using namespace sf;

class Ball
{
private:
    Vector2f m_Position;

    CircleShape m_Shape;

    float m_Speed = 1000.0f;

    float m_DirectionX = 0.2f;

    float m_DirectionY = 0.2f;

public:
    Ball(float startX, float startY) ;

    FloatRect getPosition();

    CircleShape getShape();

    void reboundSides();

    void reboundBatOrTop();

    void reboundBottom();

    void update(Time dt);

};
```

Ball.cpp

```
#include "Ball.h"
```

```
Ball :: Ball(float startX, float startY)
```

```
{  
  
    m_Position.x = startX;  
  
    m_Position.y = startY;  
  
    m_Shape.setRadius(10);  
  
    m_Shape.setPosition(m_Position);  
  
}
```

```
FloatRect Ball::getPosition()
```

```
{  
  
    return m_Shape.getGlobalBounds();  
  
}
```

```
CircleShape Ball::getShape()
```

```
{  
    return m_Shape;  
}
```

```
void Ball::reboundSides()
```

```
{  
  
    m_DirectionX = -m_DirectionX;  
  
}
```

```
void Ball::reboundBatOrTop()
{
    m_DirectionY = - m_DirectionY;
}
```

```
void Ball::reboundBottom()
{
    m_Position.y = 11;
    m_Position.x = 500;
```

```
    //m_DirectionY = - m_DirectionY;

}
```

```
void Ball::update(Time dt)
{
    m_Position.y += m_DirectionY * m_Speed * dt.asSeconds();

    m_Position.x += m_DirectionX * m_Speed * dt.asSeconds();

    m_Shape.setPosition(m_Position);
}
```

Bat.h

```
#include<SFML/Graphics.hpp>

using namespace sf;

class Bat
{
private:
    Vector2f m_Position;

    RectangleShape m_Shape;

    float m_Speed = 600.0f;

    bool m_MovingLeft = false;

    bool m_MovingRight = false;
public:
    Bat(float startX, float startY) ;

    FloatRect getPosition();

    RectangleShape getShape();

    void moveLeft();

    void moveRight();

    void stopLeft();

    void stopRight();

    void update(Time dt);

};
```

Bat.cpp

```
#include "Bat.h"
```

```
Bat :: Bat(float startX, float startY)
```

```
{  
  
    m_Position.x = startX;  
  
    m_Position.y = startY;  
  
    m_Shape.setSize(sf::Vector2f(50,5));  
  
    m_Shape.setPosition(m_Position);  
  
}
```

```
FloatRect Bat::getPosition()
```

```
{  
  
    return m_Shape.getGlobalBounds();  
  
}
```

```
RectangleShape Bat::getShape()
```

```
{  
    return m_Shape;  
}
```

```
void Bat:: moveLeft()
```

```
{  
  
    m_MovingLeft = true;  
  
}
```

```
void Bat:: moveRight()
```

```
{  
  
    m_MovingRight = true;  
  
}
```

```
void Bat::stopLeft()
```

```
{  
  
    m_MovingLeft = false;  
  
}
```

```
void Bat::stopRight()
```

```
{  
  
    m_MovingRight = false;  
  
}
```

```
void Bat::update(Time dt)
```

```
{  
    if(m_MovingLeft)  
  
        if(m_Position.x > 0)  
  
            m_Position.x -= m_Speed * dt.asSeconds();
```

```
  
    if(m_MovingRight){  
  
        if(m_Position.x < 910)  
  
            m_Position.x += m_Speed * dt.asSeconds();  
  
    }
```

```
  
    m_Shape.setPosition(m_Position);
```

```
}
```

Pong.cpp

```
#include<SFML/Graphics.hpp>

#include<sstream>

#include"Bat.h"

#include"Ball.h"

using namespace sf;

int main()

{

    VideoMode vm(960,540);

    RenderWindow window(vm,"Pong Game");

    int score =0;

    int lives = 3;

    Bat bat(960/2,540-20);

    Ball ball(960/2,0);

    Clock clock;

    Text hud;

    Font font;

    font.loadFromFile("font/KOMIKAP_.ttf");

    hud.setFont(font);

    hud.setCharacterSize(30);

    hud.setFillColor(Color::White);

    hud.setPosition(20,20);

    while(window.isOpen()){

        Event event1;

        while (window.pollEvent(event1))

        {

            if (event1.type == event1.Closed)
```

```
{  
    window.close();  
}  
  
}  
  
if (Keyboard::isKeyPressed(Keyboard::Escape))  
{  
    window.close();  
}  
  
if (Keyboard::isKeyPressed(Keyboard::Left))  
{  
    bat.moveLeft();  
}  
  
else  
{  
    bat.stopLeft();  
}  
  
if (Keyboard::isKeyPressed(Keyboard::Right))  
{  
    bat.moveRight();  
}  
  
else  
{  
    bat.stopRight();  
}
```



```
Time dt = clock.restart();
```

```
bat.update(dt);
```

```
ball.update(dt);
```

```
std::stringstream ss;
```

```
ss<<"Score "<<score<<" Lives "<<lives;
```

```
hud.setString(ss.str());
```

```
if(ball.getPosition().left<0 || ball.getPosition().left +ball.getPosition().width>window.getSize().x)
```

```
{
```

```
ball.reboundSides();
```

```
}
```

```
if(ball.getPosition().top<0)
```

```
{
```

```
ball.reboundBatOrTop();
```

```
score++;
```

```
}
```

```
if(ball.getPosition().top>window.getSize().y)
```

```
{
```

```
ball.reboundBottom();
```

```
lives--;
```

```
if(lives<1)
```

```
{
```

```
ss<<"Game Over";
```

```
    hud.setString(ss.str());

    score=0;

    lives=3;

}

}

if(ball.getPosition().intersects(bat.getPosition()))

{

    ball.reboundBatOrTop();

}

    window.clear(Color::Black);

    window.draw(hud);

    window.draw(bat.getShape());

    window.draw(ball.getShape());

    window.display();

}

    return 0;

}
```